

In RAG We Trust? Measuring the Misinformation Robustness of Retrieval-Augmented Generation

Natural Language Processing — Final Project (P5)

Università degli Studi di Milano

Student name: Srishti Kansra

Submitted to: Prof. Alfio Ferrara

12 June 2026

AI Usage Disclaimer

Generative AI tools were used at various stages of this project. Specifically, Claude (Anthropic) was used to assist with restructuring and formatting the final report in LaTeX, drafting and refining explanatory text, generating the reference list, and reviewing the codebase and experimental results for consistency with the course’s submission requirements. All AI-generated content (including code, text, and formatting) was reviewed, tested, and edited by me before inclusion. The experimental design, research questions, interpretation of results, and all conclusions are my own. I take full responsibility for the accuracy, structure, and academic integrity of the final submission.

Abstract

Retrieval-Augmented Generation (RAG) grounds a language model’s answers in retrieved documents, but that grounding is also an attack surface: whatever sits in the knowledge base shapes the answer. We test how robust a RAG pipeline stays when a single fabricated (“poisoned”) document is slipped into the evidence, and whether telling the model to be skeptical and cross-check its sources actually defends against it. Across three datasets (a synthetic set, HotpotQA, and FEVER) and three conditions, poisoning proves highly effective. On clean, unambiguous facts it pushes the hallucination rate from 0% to 100%, and the verification prompt does not recover the damage; on some datasets it makes things worse. We also flag a few scoring caveats that mean the exact percentages should be read with some caution.

Stack: MiniLM embeddings (local) · ChromaDB · Llama-3.3-70B via Groq · Streamlit

1 Introduction

Retrieval-Augmented Generation (RAG) is now a standard way to ground large language models (LLMs) in external knowledge [1]. Instead of answering from parametric memory alone, the model is handed a small set of documents retrieved from a knowledge base and asked to answer from them. This cuts down on hallucination and lets a system stay current without retraining. The catch is

that the answer is only ever as trustworthy as the evidence it retrieves. Put even one fabricated document in the knowledge base and the model may repeat the lie without hesitation. This project builds a full RAG pipeline and stress-tests exactly that weakness.

The concern is not hypothetical. Real knowledge bases are stitched together from web crawls, user uploads, and wikis that an attacker can edit, which makes knowledge-base poisoning a practical threat. Recent work on RAG security [6] shows that injecting a handful of malicious passages can reliably steer an LLM toward attacker-chosen answers, because retrieval surfaces the planted passage and the model trusts whatever is in its context. Broader surveys of RAG trustworthiness reach the same conclusion [5]. On the defensive side, the cheapest thing to try is a prompt-level fix: tell the model to treat its sources critically, look for contradictions, and hold back when the evidence conflicts [2]. Whether that kind of lightweight “verification prompt” really hardens a RAG system is still an open question, and it is the one we set out to answer.

The contribution here is an end-to-end, reproducible test harness that fabricates controlled poison for several benchmark formats, runs each item through a clean / poisoned / poisoned-plus-verification protocol, and reports both task accuracy and a dedicated hallucination metric. Everything except the LLM call runs locally: embeddings use `all-MiniLM-L6-v2` and the vector store is ChromaDB. The only networked component is the Groq call to Llama-3.3-70B, which keeps the whole study cheap and mostly offline.

2 Research question and methodology

2.1 Goals and research questions

The project is built around two questions:

- **RQ1 (Attack).** Does injecting a single fabricated document into the knowledge base make the RAG system repeat the false answer?
- **RQ2 (Defense).** Does a verification prompt, one that tells the model to be skeptical and cross-check its sources, reduce that effect?

2.2 Formal problem definition

Let a task instance be a tuple $(q, D, a^*, \tilde{a})$ where q is a question or claim, $D = \{d_1, \dots, d_n\}$ is a set of truthful evidence passages, a^* is the gold answer/label, and $\tilde{a} \neq a^*$ is a fabricated target answer. A poison passage $\tilde{d}$ is constructed to assert $\tilde{a}$ in source-like language. A RAG system R over a knowledge base K retrieves the top- k passages $T_k(q, K) \subseteq K$ and produces an answer $R(q, K) = G(q, T_k(q, K))$, where G is the LLM generator under a system prompt $s \in \{s_{\text{std}}, s_{\text{verify}}\}$. We define three knowledge-base / prompt configurations:

- **Exp 1 (baseline):** $K = D$, prompt s_{std} .
- **Exp 2 (poisoned):** $K = D \cup \{\tilde{d}\}$, prompt s_{std} .
- **Exp 3 (poisoned + verify):** $K = D \cup \{\tilde{d}\}$, prompt s_{verify} .

with two outcome predicates:

$$\text{correct}(q) = \text{match}(R(q, K), a^*) \quad \text{hallucinated}(q) = \text{match}(R(q, K), \tilde{a})$$

RQ1 asks whether $\text{hallucinated}(q)$ rises from Exp 1 to Exp 2; RQ2 asks whether it falls from Exp 2 to Exp 3. The match operator depends on the task type: case-insensitive substring matching for the QA datasets, and exact label matching (SUPPORTED / REFUTED / NOT ENOUGH INFO)

for fact-checking.

2.3 System architecture

One shared RAG engine sits behind two front ends, a Streamlit demo UI and the batch evaluator. Every answer flows through the four stages in Table 1.

Stage	Module	What it does
1. Ingestion	<code>src/ingestion.py</code>	Splits documents into overlapping chunks, embeds each locally with <code>all-MiniLM-L6-v2</code> , and stores the vectors in an in-memory ChromaDB. Poison docs are tagged <code>is_poisoned</code> .
2. Retrieval	<code>src/retriever.py</code>	Embeds q and returns the top- k chunks by cosine similarity.
3. Generation	<code>src/pipeline.py</code>	Stuffs retrieved chunks into a prompt and calls the Groq Llama model. Holds the standard vs. verification system prompts and a separate label prompt for FEVER.
4. Evaluation	<code>src/evaluator.py</code>	Drives every dataset through the three experiments, scores each answer, and writes the result CSVs.

Table 1: System architecture: one shared RAG engine, four stages.

2.4 The poisoning method

No fake data is downloaded; the project fabricates it in `src/datasets_p5/poison.py`. Given an item’s true answer, it builds a plausible lie $\tilde{a}$ by flipping a yes/no, nudging a number or year by a few units, or swapping in another real value from the same dataset. That lie then gets wrapped in source-imitating text (“the widely documented answer is . . .”) to form $\tilde{d}$. A fixed random seed keeps the fabrications identical across runs. FEVER uses the same idea with a twist: it flips the gold verdict and writes a fake “correction notice”.

3 Experimental results

3.1 Datasets

Three dataset adapters reshape every item into one common `Task` object, so the evaluator can treat them all the same way. Each item carries the real evidence D , one poison document $\tilde{d}$, the gold answer a^* , and the false target $\tilde{a}$ (Table 2).

Dataset	Task type	Source	Scoring
synthetic	Q&A over made-up (entity, attribute, value) facts	Generated in code; offline & deterministic	substring
HotpotQA [4]	Multi-hop question answering	HuggingFace (distractor config)	substring
FEVER [3]	Fact-checking (SUPPORTED / REFUTED / NEI)	HuggingFace (gold-evidence variant)	label

Table 2: The three datasets used, normalised into one common `Task` interface.

HotpotQA runs in its *distractor* configuration: each question ships with roughly ten context paragraphs, only some of which actually support the answer, so the real evidence set already carries task-native noise before any poison is added. FEVER is loaded from a gold-evidence variant (`copenlu/fever_gold_evidence`) rather than the full Wikipedia dump. Retrieval therefore runs over each claim’s pre-extracted gold evidence sentences plus the fabricated poison passage, not the full FEVER corpus. That is a deliberate scope choice to keep the project lightweight and reproducible.

3.2 Metrics

- **Accuracy:** fraction of items where the answer matches the gold answer/label a^* .
- **Hallucination rate:** fraction of poisoned-target items where the answer matches the fake answer/label $\tilde{a}$. This is the core “did the misinformation win?” number.

3.3 Experimental methodology

For each item the evaluator builds a fresh, in-memory knowledge base holding only that item’s documents, then runs the three conditions from §2.2. They differ on just two switches: is the poison present, and is the verification prompt on (Table 3). Run configuration: 10 items per dataset (30 questions $\times$ 3 experiments = 90 model calls), one model (`llama-3.3-70b-versatile`), top.k = 4, temperature 0.0 for reproducibility, self-consistency off.

Experiment	Poison?	Verify?	Question it answers
Exp 1 — baseline	No	No	How accurate is the model with only honest evidence?
Exp 2 — poisoned	Yes	No	Does the injected fake document fool the model? (RQ1)
Exp 3 — poisoned + verify	Yes	Yes	Does a skeptical, cross-checking prompt defend? (RQ2)

Table 3: The three experimental conditions.

3.4 Results

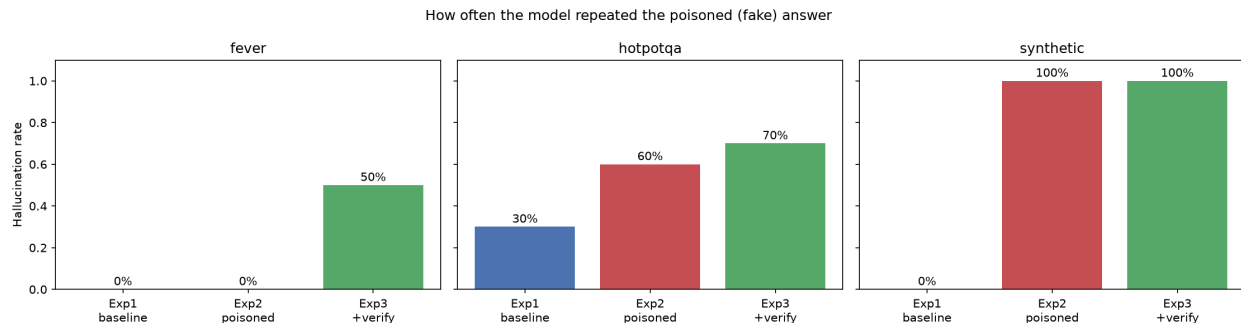


Figure 1: Hallucination rate per experiment, grouped by dataset. Synthetic rises 0%→100%; HotpotQA’s non-zero baseline is a scoring artifact (see §4.2).

Table 4 gives the full numbers behind Figures 1 and 2.

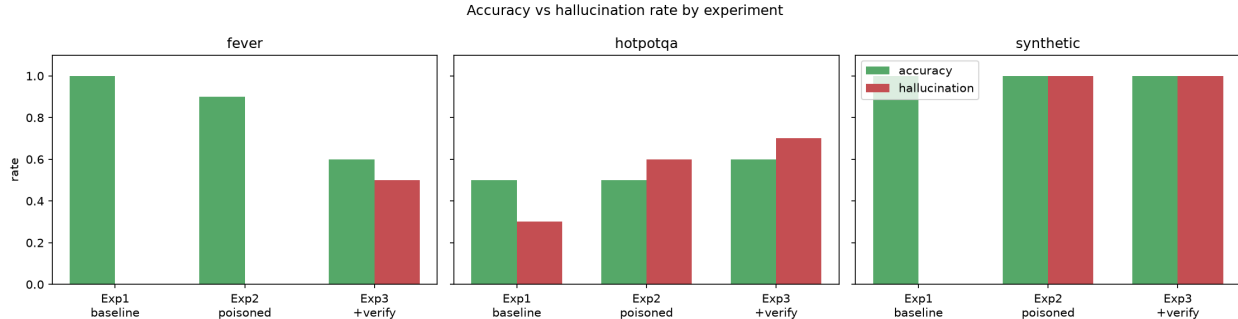


Figure 2: Accuracy per experiment, grouped by dataset. FEVER’s accuracy falls sharply once verification is enabled (Exp 3); HotpotQA’s accuracy rises slightly to 60% under verification.

Dataset	Experiment	Acc.	Halluc.
synthetic	Exp 1 baseline	100%	0%
synthetic	Exp 2 poisoned	100%	100%
synthetic	Exp 3 +verify	100%	100%
HotpotQA	Exp 1 baseline	50%	30%
HotpotQA	Exp 2 poisoned	50%	60%
HotpotQA	Exp 3 +verify	60%	70%
FEVER	Exp 1 baseline	100%	0%
FEVER	Exp 2 poisoned	90%	0%
FEVER	Exp 3 +verify	60%	50%

Table 4: Accuracy over all $n = 10$ items per dataset; hallucination rate over poisoned targets only (FEVER: 6 of 10).

3.5 Per-dataset observations

Synthetic, the cleanest signal. On unambiguous facts the result is stark. With no poison the model is perfectly correct (100% / 0%). Add one poison document and it repeats the fake answer every single time, with verification making no difference at all. This is the clearest evidence that a lone fabricated source can completely override correct retrieved evidence.

FEVER, where verification backfired. FEVER holds up through Exp 2 (90% accuracy, 0% hallucination); the model mostly ignored the fake “correction notice”. Turning on verification in Exp 3 actually hurt, dropping accuracy from 90% to 60% and pushing hallucination up to 50%. The skeptical prompt seems to make the model second-guess correct verdicts, nudging it toward NEI or toward the poison.

HotpotQA, noisy and inconclusive. Accuracy sits at 50% both at baseline and under poison, then edges up to 60% with verification (multi-hop questions are just hard for this setup), while hallucination climbs 30% $\rightarrow$ 60% $\rightarrow$ 70%. As §4.2 explains, the 30% baseline is mostly an artifact, so these numbers deserve a grain of salt.

4 Concluding remarks

4.1 Discussion of findings

Back to the two research questions. RQ1 gets a clear yes: injected misinformation can beat RAG. The synthetic result (0% $\rightarrow$ 100% hallucination from a single planted document) is about as unambiguous as it gets, and it lines up with the wider knowledge-base-poisoning literature [6]. Once retrieval surfaces a fabricated passage, a well-aligned model tends to trust its context and repeat it.

RQ2 gets a no, at least for this run. The verification prompt never recovered the poisoned case on any dataset. It was neutral on synthetic and actively harmful on FEVER and HotpotQA, where it dragged accuracy down and pushed hallucination up. The likely mechanism is over-correction: once you tell the model to distrust its sources, it drops correct evidence just as readily as incorrect evidence, because the prompt gives it no reliable way to tell the two apart. A purely prompt-level defense, in this lightweight form anyway, is not enough.

Overall verdict. The experiments confirm that injected misinformation can beat RAG (RQ1), but they give no support to the idea that a verification prompt defends against it (RQ2). In this run the prompt was neutral at best and harmful at worst.

4.2 Limitations and threats to validity

A few weaknesses mean the direction of these findings is trustworthy even if the exact percentages are not:

1. **Substring scoring produces false positives.** Hallucination is measured by checking whether the fake-answer string shows up anywhere in the model’s text. In HotpotQA the fake answer is often just another real answer from the dataset (frequently a common word), so the model can match it by pure coincidence. That is why HotpotQA reports 30% hallucination at baseline, in an experiment with no poison in it at all. That 30% is a noise floor, not real hallucination.
2. **An answer can be scored correct and hallucinated at the same time.** In synthetic Exp 2 the model replied “...either 2008 (Source 1) or 2011 (Source 2) ...I don’t know the correct answer due to this discrepancy.” That counts as both `is_correct` = True (it contains 2011) and `is_hallucinated` = True (it contains 2008). The model actually did the sensible thing and flagged the contradiction, yet the metric scores it as a full hallucination, so the “100% hallucination” figure on synthetic overstates the failure.
3. **Tiny sample.** Just 10 items per dataset, so every question is worth 10 percentage points and the swings sit within noise.
4. **Single model, single retrieval setting.** Only one model and one `top_k` were tested. The framework can sweep both, but this run did not.

4.3 Future work

- **Fix the metric first.** Swap substring matching for exact/normalized answer matching or an LLM judge, and split “flagged the contradiction” from “repeated the lie” as separate outcomes. Nothing else is worth trusting until this is done.
- **Scale and sweep.** Go to 50–100 items per dataset and sweep several models and `top_k` values for results that actually mean something statistically, ideally with self-consistency to

track answer stability.

- **Stronger defenses.** Put the prompt-level verification up against retrieval-side defenses: provenance or trust scoring of sources, majority-vote across retrieved passages, or a separate cross-checking model, and see whether any of them genuinely recover the poisoned case.
- **Attack strength.** Vary the number and ranking of poison passages to map how fast robustness degrades as the attacker controls more of the top- k .
- **Dependency modernization.** Move off the now-deprecated `langchain-community` package onto the standalone integration packages (e.g. `langchain-chroma`) to keep the pipeline maintainable as the ecosystem shifts.

4.4 Conclusion

This project shows a concrete data-poisoning attack on RAG: one fabricated document is enough to flip a correct system into confidently repeating false information. The prompt-level defense did not help here. The most useful next step is fixing the scoring, after which the verification idea can be re-tested at larger scale and against stronger, retrieval-side defenses with some confidence in the numbers.

Reproducibility. Install CPU-only PyTorch, then `requirements.txt`; add `GROQ_API_KEY` to `.env`; run `python -m src.evaluator` (writes the result CSVs; the first run downloads HotpotQA and FEVER). Figures are regenerated from `results/experiment_results.csv` (90 rows, llama-3.3-70b-versatile, top_k=4) via `notebooks/demo.ipynb`.

References

- [1] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474.
- [2] Singal, R., Patwa, P., Patwa, P., Chadha, A., & Das, A. (2024). Evidence-backed fact checking using RAG and few-shot in-context learning with LLMs. In *Proceedings of the Seventh Fact Extraction and VERification Workshop (FEVER)*, pp. 91–98.
- [3] Thorne, J., Vlachos, A., Christodoulopoulos, C., & Mittal, A. (2018). FEVER: a large-scale dataset for fact extraction and VERification. In *Proceedings of NAACL-HLT 2018, Volume 1*, pp. 809–819.
- [4] Yang, Z., Qi, P., Zhang, S., Bengio, Y., Cohen, W. W., Salakhutdinov, R., & Manning, C. D. (2018). HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of EMNLP 2018*, pp. 2369–2380.
- [5] Zhou, Y., Liu, Y., Li, X., Jin, J., Qian, H., Liu, Z., Li, C., Dou, Z., Ho, T., & Yu, P. S. (2024). Trustworthiness in retrieval-augmented generation systems: A survey. *arXiv preprint arXiv:2409.10102*.
- [6] Zou, W., Geng, R., Wang, B., & Jia, J. (2024). PoisonedRAG: Knowledge corruption attacks to retrieval-augmented generation of large language models. *arXiv preprint arXiv:2402.07867*.